MediTrack: Running Guide

How to run the backend (Go API) and frontend (React + Vite) on your machine · Status as of 1 Nov 2026 · Phase 1

1. Where the project stands: Phase 1 (Foundation) is complete

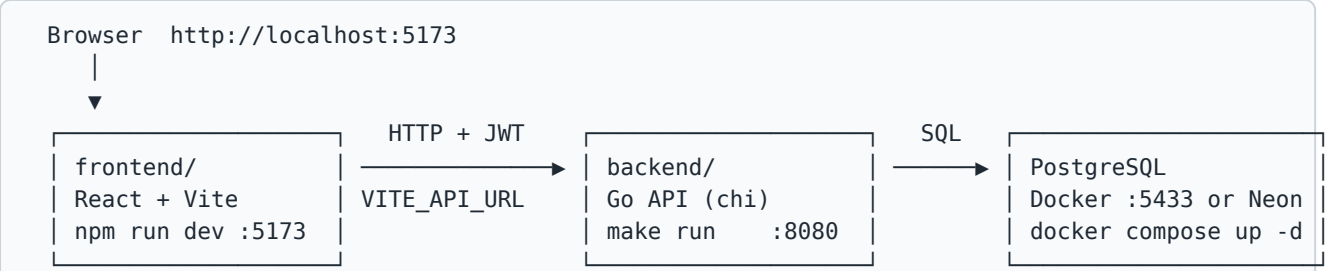
Phase 1 built the foundation everything else will sit on: login, user accounts, roles, and the full database schema. The medicine, dealer, batch and order screens and APIs are **not built yet**. Those are later phases.

Area	What exists now	Status
Database schema	4 migrations creating `users`, `dealers`, `medicines`, `batches`, `orders`, `order_items`. They run automatically when the API starts.	**Done**
Backend API (Go)	`GET /healthz`, `POST /api/auth/login`, `GET /api/auth/me`, plus admin-only `GET/POST /api/users` and `PATCH /api/users/{id}`. JWT auth, bcrypt passwords, role middleware, CORS, graceful shutdown.	**Done**
First admin seeding	An admin is created from `ADMIN_EMAIL` / `ADMIN_PASSWORD` if no active admin exists.	**Done**
Migration CLI	`go run ./cmd/migrate up | down [n] | version`	**Done**
Frontend (React)	Login page, protected routes, admin-only Users page (list, create, activate/deactivate), responsive layout with mobile menu, placeholder Dashboard.	**Done**
Tests	Go unit tests for auth (JWT/password) and middleware. All pass.	**Done**
Medicines, dealers, batches	Tables exist; no API endpoints or pages yet.	**Next**
Dashboard (expiry, low stock)	Placeholder page. The cards are planned for Phase 3.	**Later**
Purchase orders	Tables exist; no API or UI yet.	**Later**
Deployment	Render/Vercel config is sketched in the README. Full steps are planned for Phase 5.	**Phase 5**

> **Verified on 1 Nov 2026:** `go test ./...` passes, `npm run build` succeeds, and a fresh API instance against the Docker database applied migrations, seeded the admin, and logged in successfully (`/auth/me` and `/users` both returned data).

2. How the monorepo fits together

There is **no single root `npm run dev`**. The repo holds two separate programs plus a database, and each one runs in its own terminal:



Rule of thumb: **Go commands run from `backend/`, npm commands run from `frontend/`, and Docker runs from the repo root.**

3. Prerequisites (install once)

Tool	Version	Check with
Go	1.22 or newer	`go version`
Node.js + npm	20 or newer	`node -v`
Docker (for local DB)	any recent	`docker compose version`
make (optional)	any	`make --version`. Without make, use the `go …` commands shown alongside.

4. First-time setup (once per machine)

Step 1: Start the database (repo root)

```
# from MediTrack/
docker compose up -d          # Postgres 16 on localhost:5433, data kept in a Docker volume
```

Using Neon instead? Skip this step and put your Neon connection string (ending in `?sslmode=require`) in `DATABASE_URL` in Step 2.

Step 2: Configure the backend

```
cd backend
cp .env.example .env          # then edit .env
openssl rand -base64 48       # paste the output into JWT_SECRET
```

.env key	Meaning
`PORT`	API port (default 8080)
`DATABASE_URL`	Docker: `postgres://meditrack:meditrack@localhost:5433/meditrack?sslmode=disable`
`JWT_SECRET`	Long random string (32+ chars) used to sign login tokens
`TOKEN_TTL`	How long a login lasts (e.g. `12h`)
`CORS_ALLOWED_ORIGINS`	Frontend URL(s), comma-separated. Local: `http://localhost:5173`
`AUTO_MIGRATE`	`true` = apply migrations on every start
`ADMIN_NAME` / `ADMIN_EMAIL` / `ADMIN_PASSWORD`	First admin, created only if no active admin exists yet

Step 3: Configure and install the frontend

```
cd frontend
cp .env.example .env.local    # contains VITE_API_URL=http://localhost:8080
npm install
```

5. Daily workflow: run everything (3 terminals)

This replaces the single `npm run dev` you're used to. Open three terminal tabs:

Terminal 1 Database (repo root)

```
cd ~/CODE/MediTrack
docker compose up -d          # runs in background; safe to run if already up
```

Terminal 2 Backend API

```
cd ~/CODE/MediTrack/backend
make run                      # same as: go run ./cmd/api
```

Expected output: `running migrations` → (first time) `seeded first admin user` → `API listening port=8080`. Check it at `http://localhost:8080/healthz`, which should return `{"status":"ok"}`.

Terminal 3 Frontend

```
cd ~/CODE/MediTrack/frontend
npm run dev                   # http://localhost:5173
```

Log in

Open **http://localhost:5173** and sign in with the admin from `backend/.env`. The default is `admin@hospital.local` / `ChangeMe123`. Change it once you're in.

> **Order matters a little:** start the DB before the backend (the API exits if it can't reach Postgres). The frontend can start any time, but API calls fail until the backend is up.

Stopping

```
Ctrl+C                        # in Terminal 2 and Terminal 3
docker compose stop           # from repo root: stop DB, keep data
docker compose down           # remove container, keep data volume
docker compose down -v        # ⚠ also DELETES all database data
```

6. Command cheat sheet

Backend (run from `backend/`)

With make	Without make	What it does
`make run`	`go run ./cmd/api`	Start API (auto-migrates + seeds admin)
`make test`	`go test ./...`	Run Go tests
`make build`	`go build -o bin/api ./cmd/api`	Build a binary at `bin/api` (run with `./bin/api`)
`make migrate-up`	`go run ./cmd/migrate up`	Apply pending migrations
`make migrate-down`	`go run ./cmd/migrate down`	Roll back last migration (`down 4` = last 4)
`make migrate-version`	`go run ./cmd/migrate version`	Show current schema version
`PORT=8090 make run`		Run on a different port for one session

Frontend (run from `frontend/`)

Command	What it does
`npm install`	Install dependencies (after cloning or when package.json changes)
`npm run dev`	Dev server with hot reload on :5173
`npm run build`	Type-check + production build into `dist/`
`npm run preview`	Serve the production build locally
`npm run lint`	Lint with oxlint

Database (run from repo root)

Command	What it does
`docker compose up -d`	Start Postgres on :5433
`docker compose ps`	Is it running?
`docker compose logs -f db`	Follow DB logs
`docker compose exec db psql -U meditrack`	Open a SQL shell (e.g. `\dt` lists tables)

Quick API test from the terminal

```
curl localhost:8080/healthz
curl -X POST localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@hospital.local","password":"ChangeMe123"}'
# copy the "token" value, then:
curl localhost:8080/api/auth/me -H "Authorization: Bearer <token>"
```

7. Troubleshooting

Symptom	Cause and fix
`listen tcp :8080: bind: address already in use`	Another API is already running, often an old `./bin/api` from an earlier build. Find it with `ss -ltnp | grep 8080` (or `lsof -i :8080`), stop it with `kill <pid>`, then `make run` again. Or use `PORT=8090 make run`, but then set `VITE_API_URL=http://localhost:8090` too.
Code changes don't show up in the API	`bin/api` is a compiled snapshot and doesn't pick up code changes. Use `make run` during development, or run `make build` again before `./bin/api`. Go has no hot reload: press Ctrl+C and `make run` after editing Go code.
API exits with a connection refused / DB error	Postgres isn't up. Run `docker compose up -d` from the repo root and check `docker compose ps`. Confirm `DATABASE_URL` uses port **5433**.
Login says "invalid email or password"	The admin is only seeded when *no active admin exists*, so editing `ADMIN_PASSWORD` later doesn't change an existing admin. Also check that the frontend is talking to the API instance you think it is (see the port 8080 row).
Browser console shows a CORS error	`CORS_ALLOWED_ORIGINS` in `backend/.env` must exactly match the frontend URL (`http://localhost:5173`). Restart the API after changing it.
Frontend calls the wrong URL	Edit `frontend/.env.local` → `VITE_API_URL`, then restart `npm run dev`. Vite reads env files only at startup.
Port 5173 in use	Vite picks the next port (5174…). Add that URL to `CORS_ALLOWED_ORIGINS`, or free 5173.
Want a clean database	`docker compose down -v && docker compose up -d`, then `make run` (re-migrates and re-seeds admin). ⚠ Deletes all data.

8. Optional: one command for everything

To get back to a single `npm run dev`, add a tiny `package.json` at the repo root that uses `concurrently`. This hasn't been added to the repo yet; it's an optional convenience.

```
# from MediTrack/
npm init -y
npm install -D concurrently
```

```
// MediTrack/package.json → "scripts"
"scripts": {
  "db":  "docker compose up -d",
  "dev": "npm run db && concurrently -n api,web -c cyan,magenta \"make -C backend run\"
\"npm --prefix frontend run dev\""
}
```

Then from the repo root: `npm run dev` starts the DB, API and frontend together, and Ctrl+C stops both servers.

9. Quick reference card

Service	Folder	Start command	URL
Database	repo root	`docker compose up -d`	localhost:5433
Backend API	`backend/`	`make run`	http://localhost:8080 (`/healthz`)
Frontend	`frontend/`	`npm run dev`	http://localhost:5173